Лабораторна робота №8

Тема: React Fundamentals: компоненти, JSX, props, стан, події та списки.

Мета: ознайомитися з базовими принципами розробки застосунків на React. Навчитися створювати компоненти, передавати дані через props, керувати локальним станом за допомогою `useState`, обробляти події, відображати списки та використовувати умовний рендеринг.

У цій та наступних лабораторних роботах розробляється один спільний навчальний застосунок **Study Planner**. Кожна наступна лабораторна розширює попередню та додає нові можливості React.

Вимоги до звіту

1. Лабораторна робота виконується в окремому GitHub репозиторії студента або у виділеній гілці проєкту, відповідно до вимог викладача.
2. Застосунок повинен бути реалізований з використанням React та TypeScript.
3. Використовувати React 19.x та Node.js 22.22 або новішу версію.
4. Для створення UI необхідно використовувати JSX/TSX.
5. Застосунок повинен бути поділений на компоненти відповідно до їхньої відповідальності.
6. Дотримуватися єдиного стилю коду та використовувати ESLint.
7. Для стилізації використати один із підходів: SCSS із дотриманням BEM, Bootstrap або Tailwind CSS. Обраний підхід використовувати послідовно в межах роботи.
8. Не використовувати `alert` для повідомлення користувачу.
9. Код повинен бути розміщений в особистому GitHub репозиторії студента.
10. Дотримуватися Conventional Commits під час виконання роботи.

Теоретична частина

Опрацювати:

1. Quick Start — React.
2. Your First Component.
3. Writing Markup with JSX.
4. JavaScript in JSX with Curly Braces.
5. Passing Props to a Component.
6. Rendering Lists.
7. Conditional Rendering.

8. Responding to Events.
9. State: A Component's Memory.
10. Updating Objects in State.
11. Updating Arrays in State.

Практична частина

Стилізація та BEM

Для стилізації обрати один із трьох підходів:

1. **SCSS + BEM** — власні стилі з класами за схемою Block__Element—Modifier;
2. **Bootstrap** — готові utility та component класи;
3. **Tailwind CSS** — utility-first класи.

Не потрібно поєднувати всі три підходи одночасно.

Якщо обрано **SCSS + BEM**:

- стилі організувати за компонентами або логічними блоками;
- назви класів повинні відповідати BEM;
- не прив'язувати стилі до глибокої структури DOM;
- уникати глобальних стилів, які випадково впливають на інші компоненти.

Git workflow

Рекомендується виконувати роботу у окремій feature-гілці, наприклад `feature/react-fundamentals`.

Приклад Conventional Commit:

```
feat(lab-08): implement study planner basics
```

Гілку та назву commit можна змінити відповідно до правил курсу, але історія повинна показувати логічні етапи роботи.

Постановка задачі

Створити клієнтський застосунок **Study Planner** — простий планувальник навчальних завдань.

На першому етапі застосунок працює лише з даними, що зберігаються в пам'яті React. Після перезавантаження сторінки дані можуть скидатися. Збереження у `localStorage`, Context API,

`useReducer`, REST API, Router та інші можливості будуть додані в наступних лабораторних роботах.

Очікуваний основний сценарій:

1. користувач бачить список навчальних завдань;
2. може додати нове завдання;
3. може позначити завдання як виконане;
4. може видалити завдання;
5. бачить загальну кількість завдань та кількість виконаних;
6. може переглядати лише всі або лише невиконані/виконані завдання.

1. Створення React-проєкту

1. Створити новий проєкт Vite + React + TypeScript.
2. Переконатися, що проєкт використовує React 19.x та Node.js 22.22 або новішу версію.
3. Налаштувати запуск застосунку через npm.
4. Переконатися, що TypeScript та ESLint працюють без помилок.
5. Ознайомитися зі структурою стартового проєкту.
6. Видалити демонстраційний код, який не потрібний для лабораторної роботи.

Проєкт має бути створений на основі Vite + React.

2. Модель навчального завдання

Описати тип даних `Task` за допомогою TypeScript.

Завдання повинно містити щонайменше:

- унікальний `id`;
- `title` — назву;
- `description` — короткий опис;
- `completed` — ознаку виконання;
- `priority` — пріоритет;
- `category` — категорію.

Допустимі значення пріоритету: `low`, `medium`, `high`.

Приклад даних:

```
const initialTasks: Task[] = [
  {
    id: 1,
```

```
    title: 'React Components',
    description: 'Опрацювати базові принципи компонентів React',
    completed: false,
    priority: 'high',
    category: 'React',
  },
  {
    id: 2,
    title: 'TypeScript Exercises',
    description: 'Виконати практичні вправи',
    completed: true,
    priority: 'medium',
    category: 'TypeScript',
  },
];
```

3. Компоненти застосунку

Розділити застосунок щонайменше на такі компоненти:

- `App`;
- `Header`;
- `TaskForm`;
- `TaskList`;
- `TaskItem`;
- `TaskFilters`;
- `TaskStats`.

Компоненти повинні взаємодіяти через props та callback-функції.

Не слід зберігати всю логіку в одному файлі `App.tsx`.

4. Відображення списку завдань

Створити компонент `TaskList`, який отримує список завдань через props та відображає його за допомогою `map`.

Для кожного елемента необхідно використовувати стабільний `key`.

Компонент `TaskItem` повинен відображати:

- назву;
- опис;
- категорію;
- пріоритет;

- стан виконання;
- кнопки керування.

Для виконаного завдання передбачити помітну візуальну відмінність.

5. Додавання завдання

У компоненті `TaskForm` реалізувати форму з полями:

- Назва;
- Опис;
- Категорія;
- Пріоритет.

При натисканні кнопки «Додати» нове завдання повинно з’явитися у списку.

Після успішного додавання форма повинна очиститися.

Поле назви повинно бути обов’язковим. Порожнє завдання додавати не можна.

Валідація у цій лабораторній може бути простою та виконуватися без сторонніх бібліотек.

6. Зміна стану завдання

Для кожного завдання реалізувати можливість:

1. позначити завдання виконаним;
2. повернути завдання у стан «не виконано»;
3. видалити завдання.

Оновлення масиву завдань повинно виконуватися незмінно (immutable update), без прямої зміни існуючого масиву або об’єкта.

7. Фільтрація завдань

У компоненті `TaskFilters` реалізувати три режими:

- Всі;
- Виконані;
- Не виконані.

При зміні фільтра список на екрані повинен змінюватися без зміни вихідного масиву завдань.

Активний фільтр повинен бути візуально позначений.

8. Статистика

Створити компонент `TaskStats`, який показує:

- загальну кількість завдань;
- кількість виконаних;
- кількість невиконаних.

Статистика повинна автоматично оновлюватися після додавання, видалення або зміни стану завдання.

Для отримання кількості використайте поточний масив завдань та звичайні JavaScript-операції над масивами, наприклад `filter()` або `length`. На цьому етапі не потрібно використовувати окремий state для кожного лічильника.

9. Умовний рендеринг

Реалізувати повідомлення для порожнього списку.

Наприклад:

```
Завдань поки немає.
Додайте перше навчальне завдання.
```

Для порожнього результату поточного фільтра виводити окреме повідомлення, наприклад:

```
За цим фільтром завдань немає.
```

Кнопка видалення повинна бути доступна лише там, де це логічно потрібно, а виконане завдання повинно мати інший візуальний стан.

10. Оформлення

Створити простий, акуратний інтерфейс.

Рекомендована структура:

```
Study Planner
 ├──── Header
 ├──── TaskForm
 ├──── TaskStats
 ├──── TaskFilters
 └──── TaskList
     ├──── TaskItem
     ├──── TaskItem
```

```
    └──── ...
```

Не потрібно відтворювати складний дизайн. Головна мета цієї лабораторної — продемонструвати роботу React-компонентів та стану.

11. Додаткові завдання

За бажанням реалізувати один або декілька додаткових пунктів:

1. Пошук завдань за назвою.
2. Сортування за пріоритетом.
3. Лічильник кількості завдань безпосередньо у кнопках фільтра.
4. Кнопку «Очистити виконані».
5. Вибір кольору або іконки для категорії.
6. Темну тему на рівні UI.

Додаткові можливості не повинні замінювати виконання основних вимог.

Рекомендована структура

```
src/
 ├──── components/
 │    ├──── Header.tsx
 │    ├──── TaskForm.tsx
 │    ├──── TaskFilters.tsx
 │    ├──── TaskItem.tsx
 │    ├──── TaskList.tsx
 │    └──── TaskStats.tsx
 ├──── types/
 │    └──── task.ts
 ├──── data/
 │    └──── initialTasks.ts
 ├──── App.tsx
 ├──── main.tsx
 └──── ...
```

Структура може бути змінена студентом, якщо збережено логічний поділ відповідальності компонентів.

На цьому етапі не потрібно будувати складну архітектуру. Основна увага — зрозумілим компонентам, props, state та подіям.

Не потрібно використовувати feature-first, FSD, Repository Pattern, глобальні state managers або інші складні архітектурні підходи.

Демонстрація на захисті

За час захисту необхідно продемонструвати:

1. додавання нового завдання;
2. перемикання завдання між станами «виконано» / «не виконано»;
3. роботу одного з фільтрів;
4. компонентну структуру та у коді показати використання props і `useState`.

Не потрібно демонструвати всі додаткові завдання.

Контрольні запитання

1. Що таке React-компонент і чому інтерфейс у React будується з компонентів?
2. Що таке JSX та чим JSX відрізняється від звичайного HTML?
3. Чому JSX повинен повертати один кореневий елемент або Fragment?
4. Як у JSX вставляється JavaScript-вираз?
5. Що таке props? Як вони передаються від батьківського компонента до дочірнього?
6. Чи може дочірній компонент напряму змінювати props? Як правильно передати callback для зміни даних?
7. Для чого використовується `useState`? Що повертає цей hook?
8. Чому не слід напряму змінювати масив або об’єкт, який зберігається у state?
9. У чому різниця між `setState(value)` та `setState(prev ⇒ ⋯)`?
10. Як React рендерить список елементів через `map`?
11. Для чого потрібен prop `key` у списку? Чому використання індексу масиву як key може бути проблемним?
12. Що таке умовний рендеринг у React? Назвіть декілька способів його реалізації.
13. Чим відрізняється локальний state компонента від props?
14. Де доцільно зберігати state, який використовується кількома дочірніми компонентами?
15. Що відбудеться після виклику setter-функції, яку повертає `useState`?
16. Чому статистику кількості виконаних завдань не обов’язково зберігати в окремому state?
17. Що означає immutable update і як він застосовується до масивів у React?